

Sporadic Server

Milestone 1: Technical Understanding and Research Framing

Name: Faysal Ahammed Tonmoy
Matriculation No.: 1230473
Course: Real-Time Systems

Purpose of this document

This document is my short learning note for the topic **Sporadic Server**. It is not the final seminar paper. The aim is to write down what I have understood so far before writing the final paper.

The main reference recommended for this topic is Buttazzo's real-time systems book. At the moment, I do not have direct access to that book. Therefore, I am not claiming that I used any exact explanation or page from it. For this first milestone, I used accessible material for initial understanding, especially the original paper on Sporadic Server and lecture material about real-time scheduling servers.

1 What I understood about the problem

A real-time system is not only about getting a correct result. The result also has to arrive at the correct time. For example, in a robot or controller, a late motor command or late sensor reaction may be a problem even if the calculation itself is correct.

In such systems, there are usually periodic tasks and irregular tasks. Periodic tasks come again and again after a fixed time. For example, a sensor may be read every 10 ms, or a control loop may run every 20 ms. These tasks are easier to plan because their timing is known.

Aperiodic or sporadic tasks are different. They do not arrive at a fixed time. Examples are a network command, alarm, diagnostic request, button press, or emergency event. The system should react to them quickly, but they should not disturb the important periodic tasks.

The problem is that if irregular jobs are only served when the processor is free, their response time may be bad. But if they are given too much priority, they may take processor time away from hard real-time periodic tasks. So the main question is:

How can a real-time system respond quickly to irregular requests while still protecting periodic real-time tasks?

This is the problem that server-based methods try to solve.

2 Main idea of Sporadic Server

My current understanding is that a Sporadic Server is like a controlled task that is used to serve aperiodic jobs. It does not give unlimited processor time to aperiodic work. Instead, it has a fixed amount of execution time, called server capacity or budget.

The server can run aperiodic jobs only while it still has budget. When it uses some of the budget, that amount is removed from the available budget. The used amount is restored later according to a replenishment rule.

A simple way to think about it is like mobile phone credit. If credit is available, the server can spend it to serve an aperiodic job. If the credit is used up, the server has to wait until credit is added again. Because of this, aperiodic jobs can be served quickly, but they cannot use the CPU forever.

The important point I learned is that the replenishment is not just a simple reset at every period. The consumed budget is replenished later in a controlled way. This makes the Sporadic Server different from simpler methods.

3 Basic notation I learned

A periodic task is often written as

$$\tau_i = (C_i, T_i, D_i)$$

where C_i is the worst-case execution time, T_i is the period, and D_i is the relative deadline.

The utilization of a task is

$$U_i = \frac{C_i}{T_i}$$

This tells how much processor time the task needs compared with its period.

The Sporadic Server can be written as

$$SS = (C_s, T_s)$$

where C_s is the server capacity and T_s is the replenishment period. The server utilization is

$$U_s = \frac{C_s}{T_s}$$

For example, if $C_s = 4$ ms and $T_s = 20$ ms, then

$$U_s = \frac{4}{20} = 0.2 = 20\%$$

This means that the server reserves a controlled part of the processor for irregular jobs. It does not mean that this amount is always used. It only means that the server is allowed to use this amount when aperiodic work arrives.

4 How I understand the algorithm

The algorithm can be understood in simple steps.

1. The server starts with some available budget.
2. When an aperiodic job arrives, the server checks if budget is available.
3. If budget is available, the server executes the job.
4. While the job runs, the budget decreases.
5. The used budget is planned for replenishment later.
6. If the budget becomes zero, new aperiodic jobs must wait.
7. When the replenishment time arrives, the used budget is restored.

So the server helps aperiodic jobs by giving them a chance to run earlier. At the same time, it protects periodic tasks because the server has a limit. It cannot continuously take processor time.

A simple example helped me understand this. Suppose the server has $C_s = 4$ ms and $T_s = 20$ ms. If an aperiodic job arrives at time 3 ms and uses 2 ms, then only 2 ms budget remains. The used 2 ms will be replenished later. Until then, the server cannot behave as if the budget was never used.

This is the part I still need to check more carefully from sources: the exact formal replenishment rule.

5 Comparison with other server methods

I also learned that Sporadic Server is easier to understand when compared with Polling Server and Deferrable Server.

In a Polling Server, the server checks for aperiodic work at certain times. If no aperiodic job is ready at that time, the capacity may be wasted. This can make response time bad if a job arrives just after the polling time.

In a Deferrable Server, unused budget can be kept within the current period. This improves response time, because a job arriving later in the period can still use the remaining budget. However, this preserved budget can create extra interference for periodic tasks.

The Sporadic Server tries to keep the good response behavior, but it controls the interference more carefully by replenishing consumed budget later. This is why it is more useful, but also more complex to implement.

6 Assumptions and limitations

From my current understanding, the basic discussion of Sporadic Server assumes a single processor, preemptive scheduling, known execution times, and correct priority assignment. It also assumes that the server budget and replenishment are implemented correctly.

The method has some limitations. If the server capacity is too large, periodic tasks may suffer. If the capacity is too small, aperiodic jobs may still wait too long. If the replenishment period is too long, the server may become slow to recover budget. If it is too short, the system may have more timer and scheduling overhead.

The most important limitation for me is implementation complexity. The system must track how much budget was used and when it should be replenished. If this is implemented wrongly, the timing guarantee may no longer be valid.

7 Runtime and memory aspects

The Sporadic Server needs some extra work from the system. The scheduler must check the remaining budget, update the budget while the server runs, and manage replenishment events. It also needs to store waiting aperiodic jobs and information about future replenishments.

The memory overhead is probably not very large for a small example, but it still matters in embedded systems. The runtime overhead can also matter because timers, queues, and context switches cost time. So the server improves responsiveness, but this improvement is not free.

8 Why this is relevant for embedded systems

This topic is relevant for embedded systems because many embedded devices have both regular control tasks and irregular events. For example, a robot may read sensors periodically and update motors periodically. At the same time, it may receive an emergency command, network message, or obstacle event at an unknown time.

Without a server, such irregular jobs may wait too long. With a Sporadic Server, the system can reserve a controlled amount of CPU time for them. This helps the system react faster while still protecting the periodic control tasks.

This is why I think the topic is useful for real-time embedded systems, robotics, automotive controllers, and communication-based control systems.

9 Open questions

These are the points I still need to clarify:

- How detailed should I explain the exact replenishment rule in the final 5-page paper?
- Should I compare Sporadic Server deeply with Deferrable Server, or only briefly?
- Should I use only simple utilization reasoning, or also response-time analysis?
- Is a small C/C++ simulation acceptable, or should I use UPPAAL?
- Can I use accessible lecture material for Milestone 1 until I get access to Buttazzo?

References and source-use notes

References

- [1] B. Sprunt, L. Sha, and J. P. Lehoczky, “Aperiodic Task Scheduling for Hard-Real-Time Systems,” *Real-Time Systems*, vol. 1, no. 1, pp. 27–60, 1989.
Use: main source for the original Sporadic Server idea.
- [2] S. Henkler, “Real-Time Systems Seminar Slides,” Hochschule Hamm-Lippstadt, 2026.
Use: seminar requirements, scientific writing rules, reference notes, plagiarism warning, and GitHub workflow.
- [3] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, Springer.
Use: recommended textbook reference. Current note: not directly accessible at this stage, so no specific claims or page numbers from this book are used here.